

IA generativa

Marta Caro Martínez

IA generativa

- Rama de la IA cuyo objetivo es la **creación de contenido nuevo**
- Diferencia con otras ramas de la IA:
 - Otras ramas: analizan y predicen
 - IA generativa: produce contenido nuevo
 - Texto, imágenes, vídeo, código, sonidos...
- Pasos:
 1. Se hace **entrenamiento** del modelo (mejor cuanto más grande sea el dataset)
→ el modelo aprende conocimiento general
 2. Fine-tuning: entrenamiento con un dataset más pequeño y específico a la tarea
 3. Se **genera el contenido** a petición del usuario (prompts) gracias al aprendizaje del modelo
- Algoritmo básico en IA generativa: **transformers**

Origen de la IA generativa

- **Origen de la IA generativa unido al PLN**
- **Procesamiento de Lenguaje Natural (PLN):** rama de la IA que aprende del lenguaje humano (en distintos idiomas) para hacer predicciones o generar textos
 - Clasificar, traducir, responder preguntas (QA), análisis de sentimientos...
- No es lo mismo que IA generativa:
 - PLN: aprende y entiende contenido textual
 - IA generativa: crea contenido (incluyendo texto)

Origen de la IA generativa

- Origen del PLN:
 1. Sistemas basados en reglas en los años 60 → patrones predefinidos
 2. Modelos clásicos: árboles de decisión, redes bayesianas, regresión logística... → relaciones entre palabras o frases (no sobre todo el texto)
 3. Word embeddings (a partir de 2013): representación de palabras en vectores numéricos
 - Representan información semántica. Ejemplo: *perro* y *gato* tendrán valores más parecidos que *perro* y *estuche*.

Origen de la IA generativa

- Desarrollo de PLN actual:
 4. Transformers (2017) (Artículo “Attention is all you need”)
 - Analiza la relación entre todas las palabras de una secuencia → **incluye información contextual**
 - **Origen del aprendizaje por transferencia:** entrenamiento general + fine-tuning
 5. Large Language Models (LLMs) - Modelos generativos multipropósito (a partir de 2018):
 - Modelos que sirven para resolver muchas tareas
 - modelos con millones y miles de millones de parámetros → mayor capacidad para aprender patrones complejos.
 - transformers en múltiples capas
 - preentrenados en gigantescos conjuntos de datos (internet, libros, código...)
 6. Desarrollo importante de modelos multimodales (a partir de 2021)

Desafíos actuales y futuro

- Problemas principales en IA generativa: sesgos, alucinaciones, inconsistencias lógicas
- Investigación futura:
 - Arquitecturas más eficientes
 - Más desarrollo en modelos multimodales
 - Explicaciones y reducción de sesgos

GenAI VS LLMs VS Transformers

- **Transformers:** arquitectura de Deep Learning (redes neuronales)
- **LLMs:** modelos de IA muy grandes que generalmente usan Transformers como arquitectura
 - Especializados en texto
- **IA generativa:** rama de la IA para crear contenido, no tienen por qué usar LLMs, aunque generalmente lo hacen
 - Otro ejemplo de IA generativa: GANs

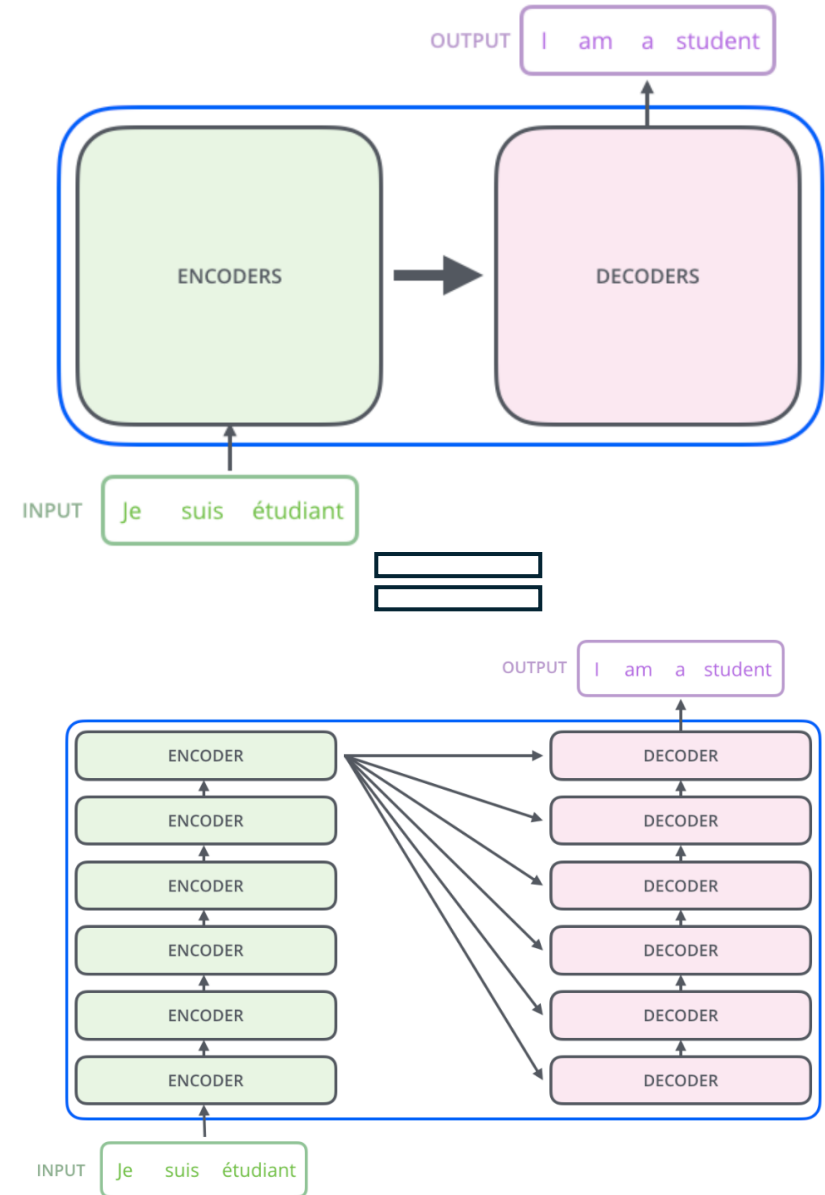
Transformers

Transformers

- Son una arquitectura de Deep Learning que tiene un gran rendimiento en tareas de **procesamiento de lenguaje natural** y **visión por computador**
- No utiliza convolución o recurrencia
- Se basa en el concepto de **mecanismo de atención**:
 - Significa que damos más **importancia a partes específicas de nuestros datos** (palabras en un texto, o regiones en una imagen) que nos interesan para obtener una predicción
 - Más eficiencia a la hora de generar predicciones

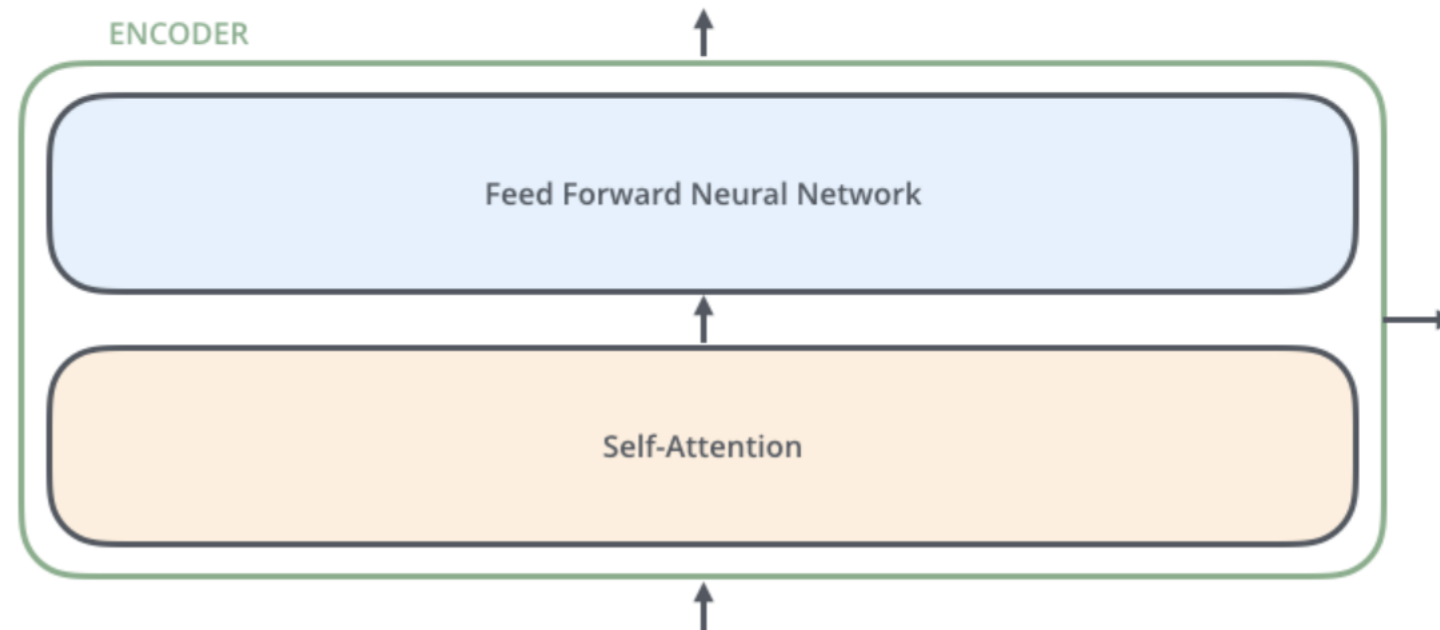
Transformers

- **Encoders:** procesan la **secuencia de entrada** y la convierten en una representación abstracta que captura la información relevante de la entrada
- **Decoders:** generan la **secuencia de salida** a partir de la representación codificada generada por el último encoder



Encoders y decoders en transformers

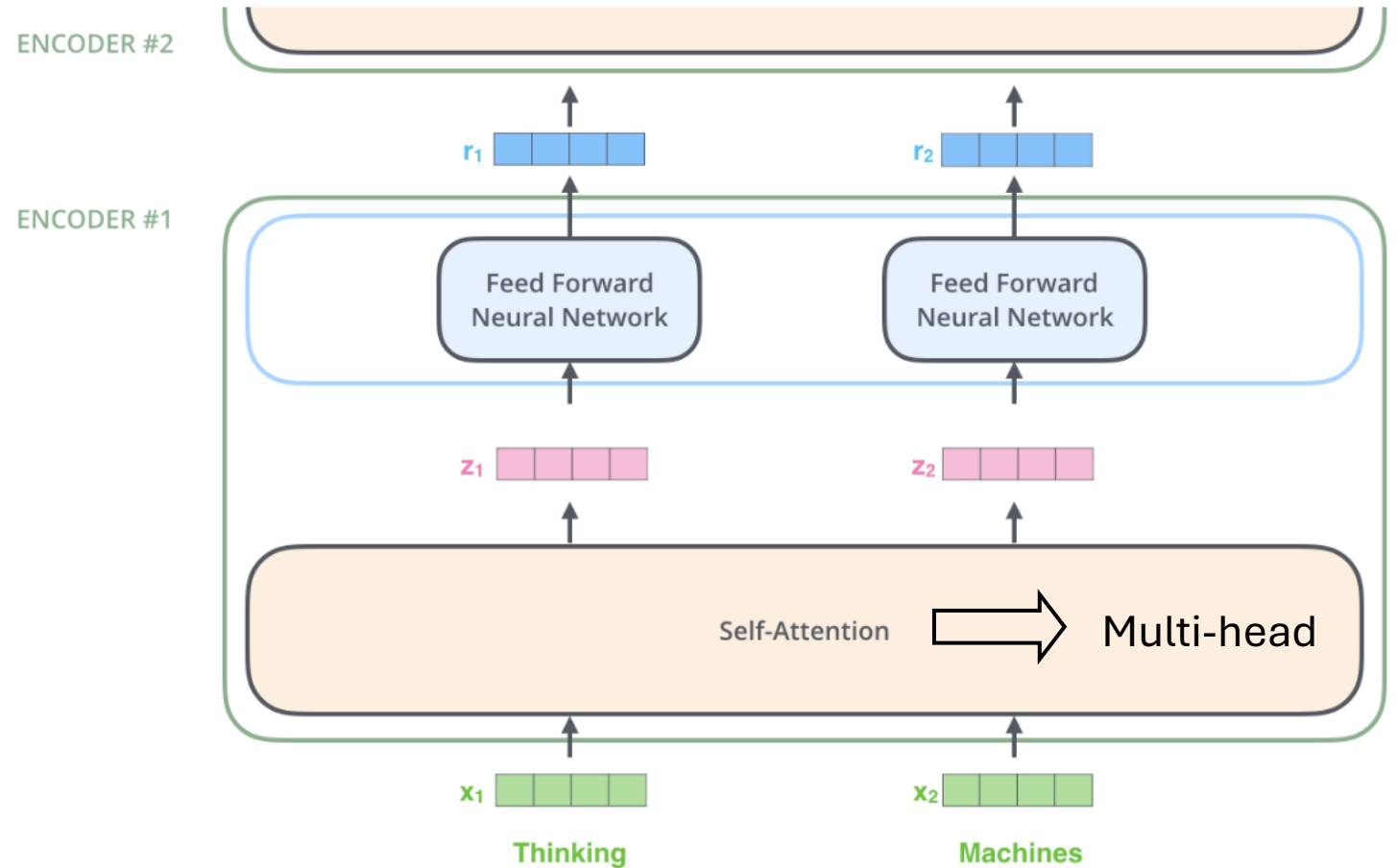
- **Self-Attention layer:** mira cuales son los elementos importantes (ej: palabras)
- **FNN layer:** para capturar relaciones no lineales



Encoders en transformers

Recibe como entrada
embedding with
signal:

- **word-embeddings + positional encoding**



Word-embeddings

- Son representaciones semánticas en forma numérica (vectores)

	Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
Gender	-1	1	-0.95	0.97	0.00	0.01
Royalness	0.01	0.02	0.93	0.95	-0.01	0.00
Edad	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.05	0.01	0.02	0.01	0.95	0.97
...	...	...	...	...	...	...

e_i es el vector columna i-esimo de la matriz word-embedding

- De esta forma, se pueden comparar palabras:

“Man” es a “Woman” $\sim e_{5391} - e_{9853} \approx [-2 \ 0 \ 0 \ 0 \ \dots]$

$$e_{man} - e_{woman} \approx e_{king} - e_w$$

“King” es a “Queen” $\sim e_{4914} - e_{7157} \approx [-2 \ 0 \ 0 \ 0 \ \dots]$

Positional encoding

- Representaciones de la posición de las palabras
- Utilizan funciones de seno (posiciones pares) y coseno (impares) para hacer los cálculos (ambos para asegurar que se representan las posiciones de forma más exacta)

El factor 10 es personalizable

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$
$$PE(pos, 2i + 1) = \cos(pos/10000^{2i/d_{model}})$$

donde:

- pos es la posición de la palabra en la secuencia
- i es la posición en el vector de positional encoding
- d_{model} es la dimensión del espacio de embedding

Positional Encoding Matrix for $d=4$ $n=100$

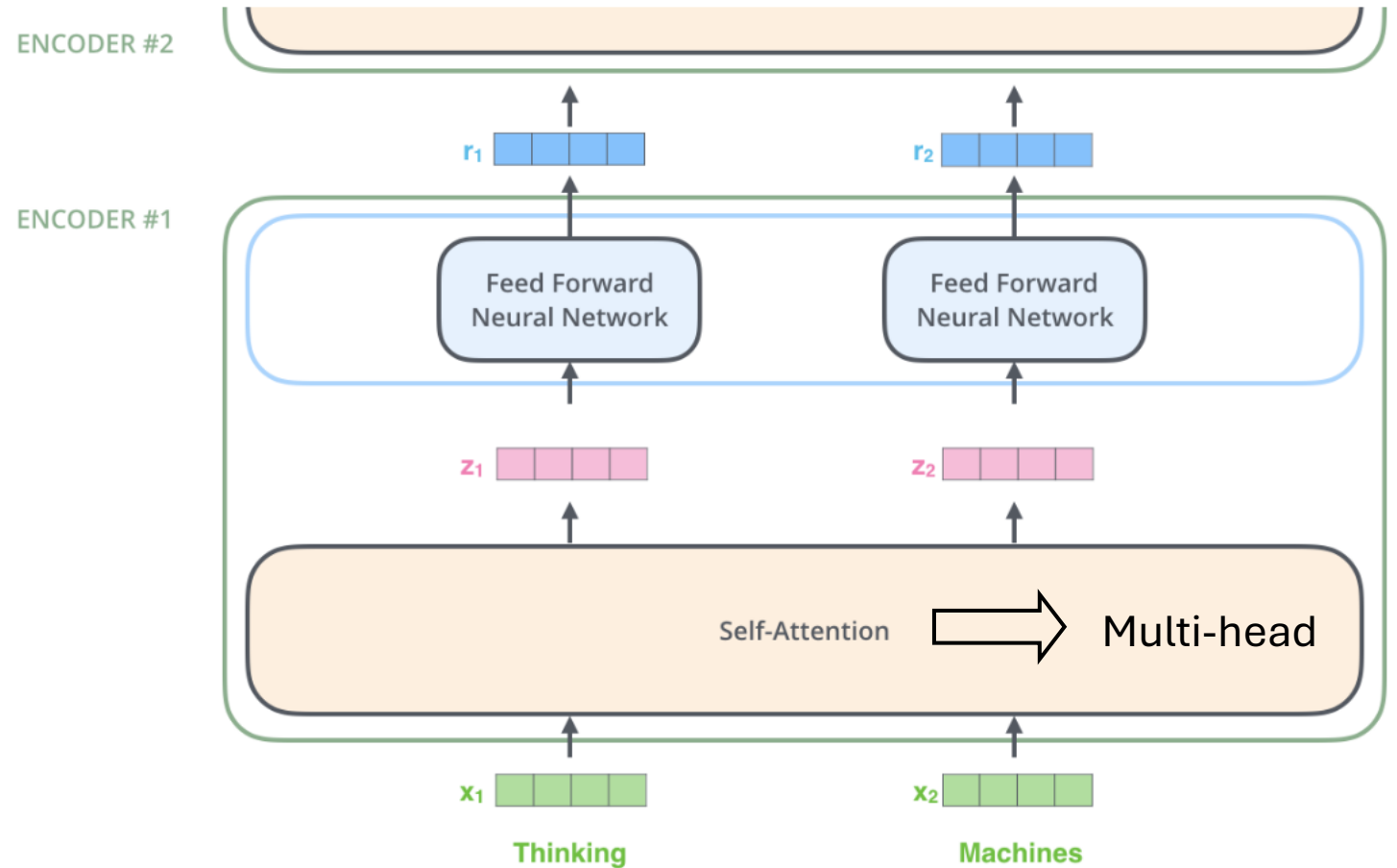
	k	i=0	i=1	i=2	i=3
The	0	$P00=\sin(0)=0$	$P01=\cos(0)=1$	$P02=\sin(0)=0$	$P03=\cos(0)=1$
dog	1	$P10=\sin(1/1)=0$	$P11=\cos(1/1)=0.54$	$P12=\sin(1/10)=0.10$	$P13=\cos(1/10)=1.0$
ran	2	$P20=\sin(2/1)=0.91$	$P21=\cos(2/1)=-0.42$	$P22=\sin(2/10)=0.20$	$P23=\cos(2/10)=0.98$
fast	3	$P30=\sin(3/1)=0.14$	$P31=\cos(3/1)=-0.99$	$P32=\sin(3/10)=0.30$	$P33=\cos(3/10)=0.96$

k es pos

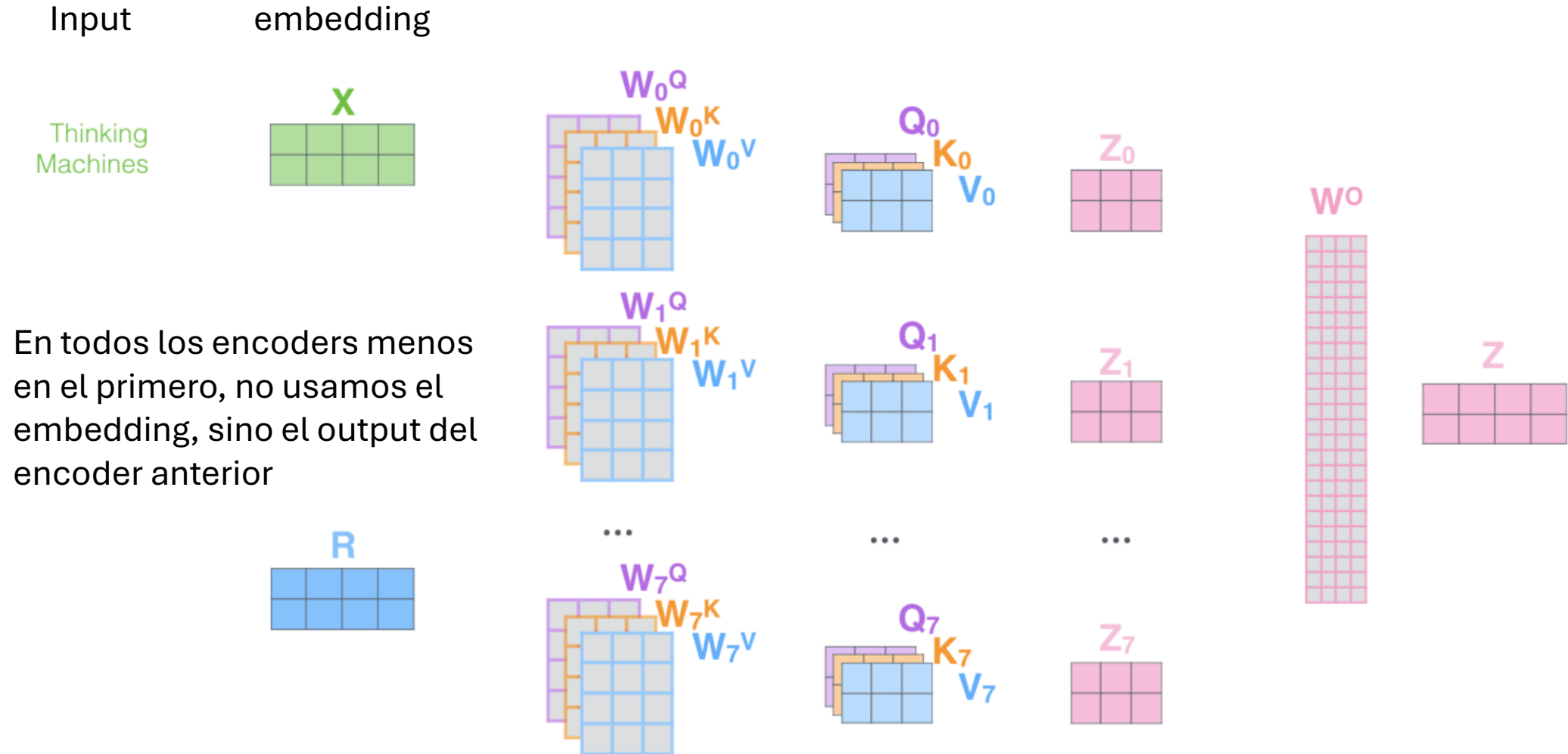
Encoders en transformers

Recibe como entrada embedding with signal:

- **word-embeddings + positional encoding**

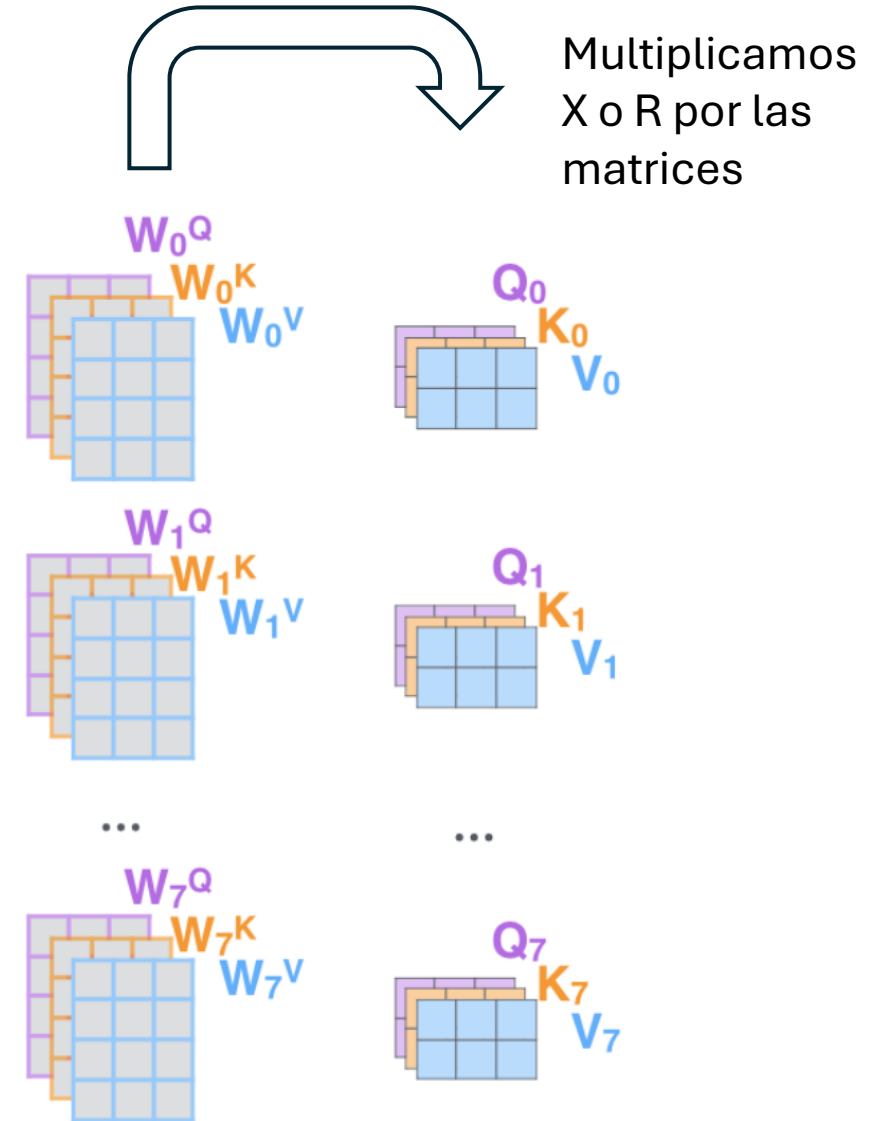


Self Attention (Multi-Head Attention)

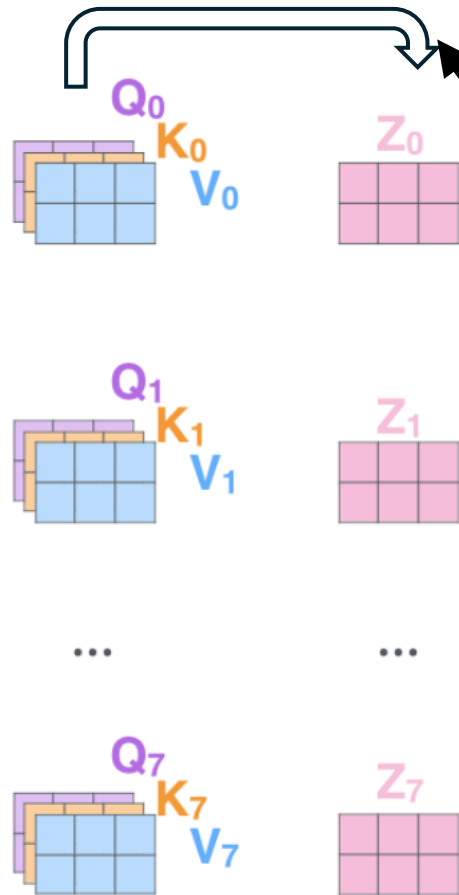


Multi-head Attention

- Creamos **8 tripletas (cabezas) de matrices de pesos** (distintos puntos de vista)
- Multiplicamos X (embedding) o R (salida del último encoder) por cada matriz de peso
- **Obtenemos 8 tripletas de matrices Q** (query), K (key), V (value)
 - Ejemplo: el modelo está procesando la palabra gato que aparece en la oración: El gato maúlla
 - $Q \rightarrow$ ¿A qué palabras debo prestar atención al procesar gato?
 - $K \rightarrow$ ¿Qué tan importante es la palabra gato en la oración?
 - $V \rightarrow$ Información que aporta gato en la oración



Self-attention



- Condensamos toda la información en una única matriz por cada cabeza

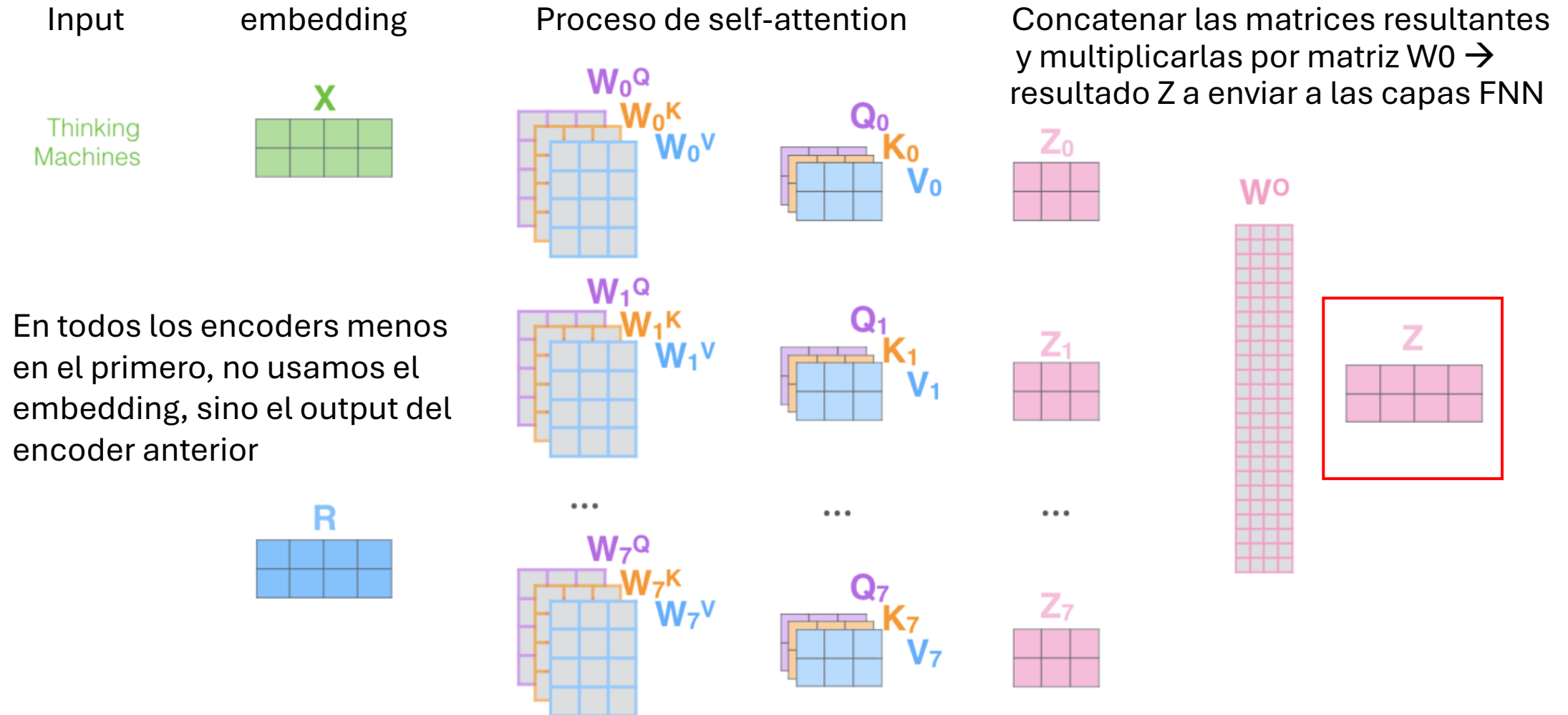
$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) V$$

La raíz cuadrada del tamaño del vector key

= Z

Resultado para cada cabeza

Self Attention (Multi-Head Attention)

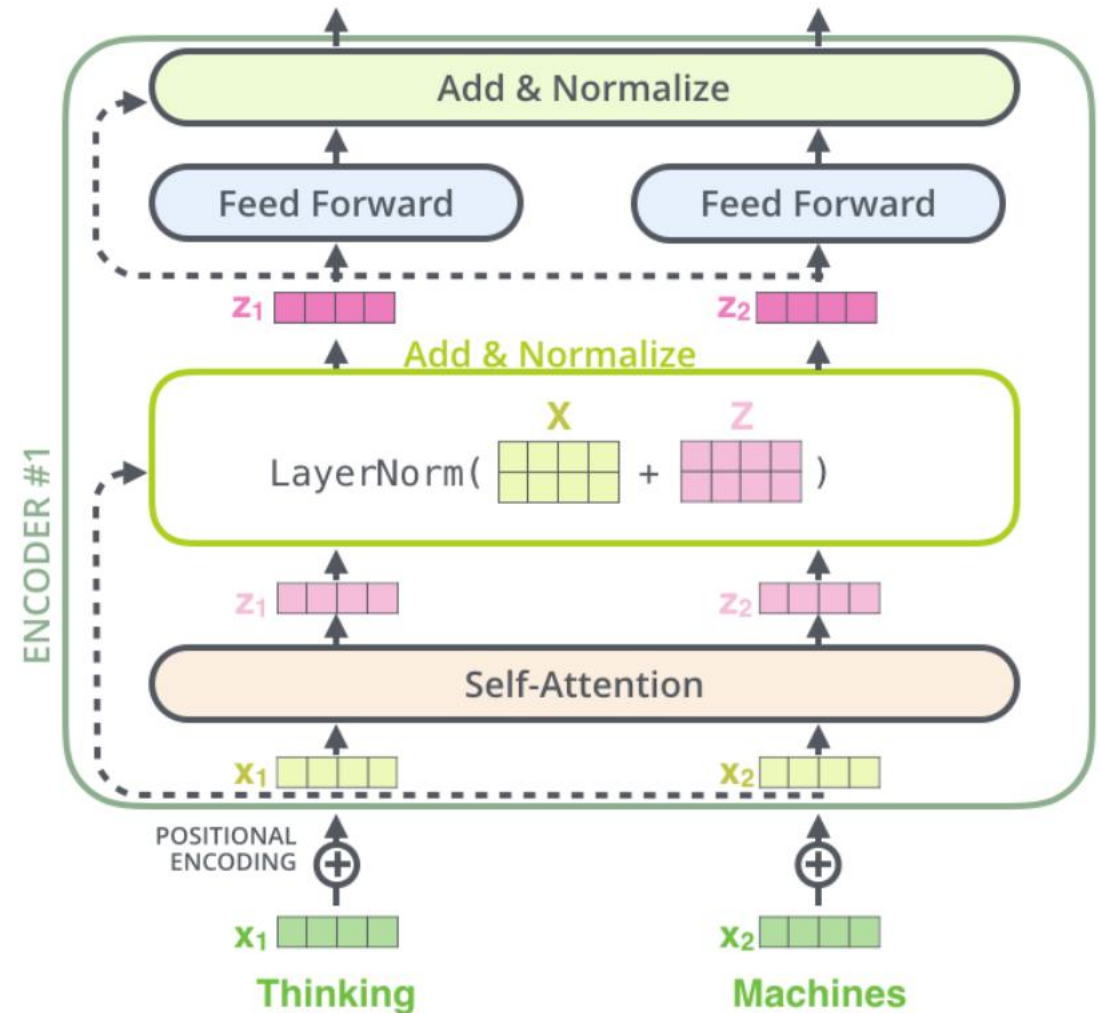


Normalización en encoders

Normalizar valores

después de las capas self-Attention y FNN

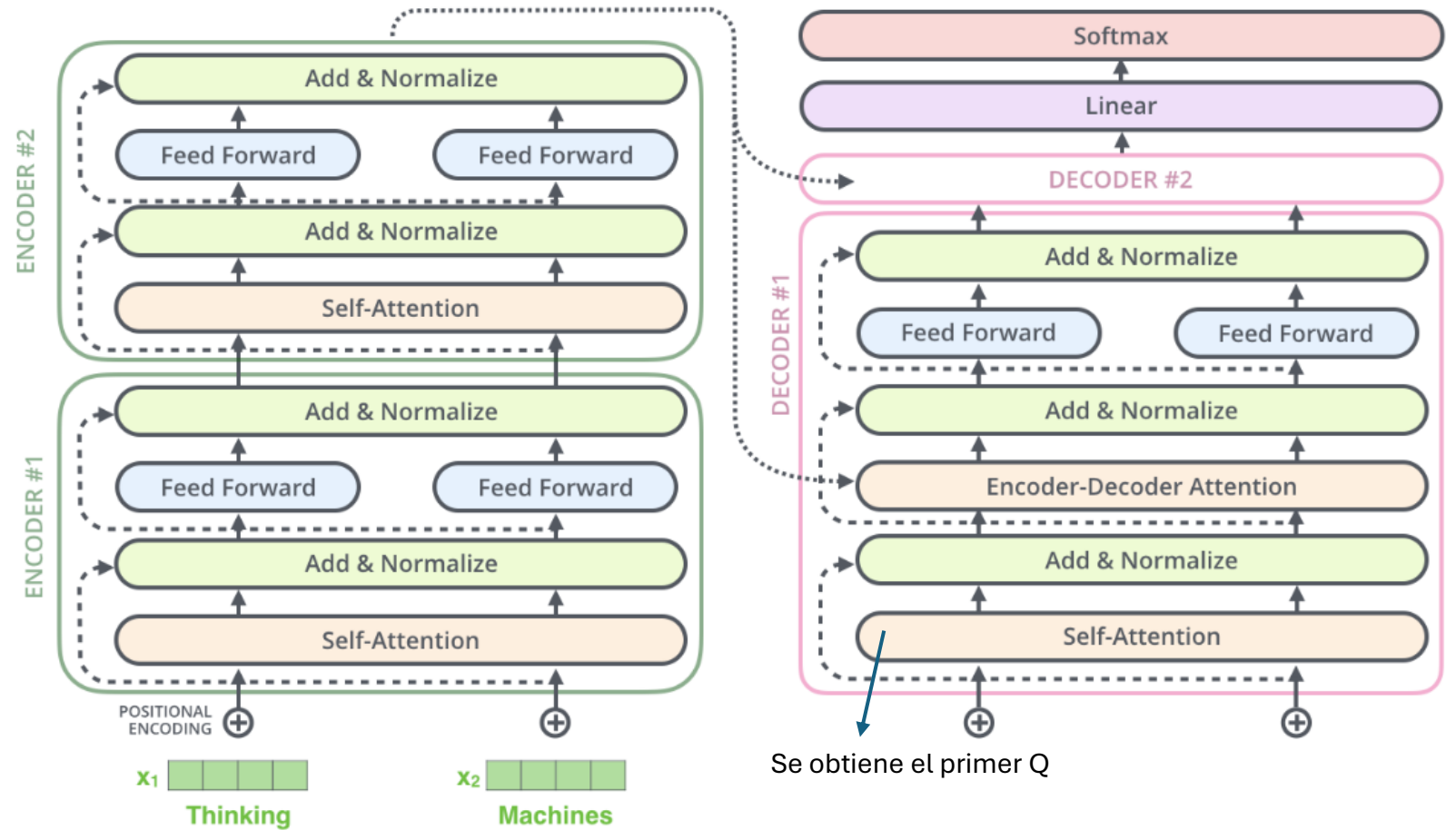
- Mejora la generalización y el entrenamiento



Encoders-Decoders

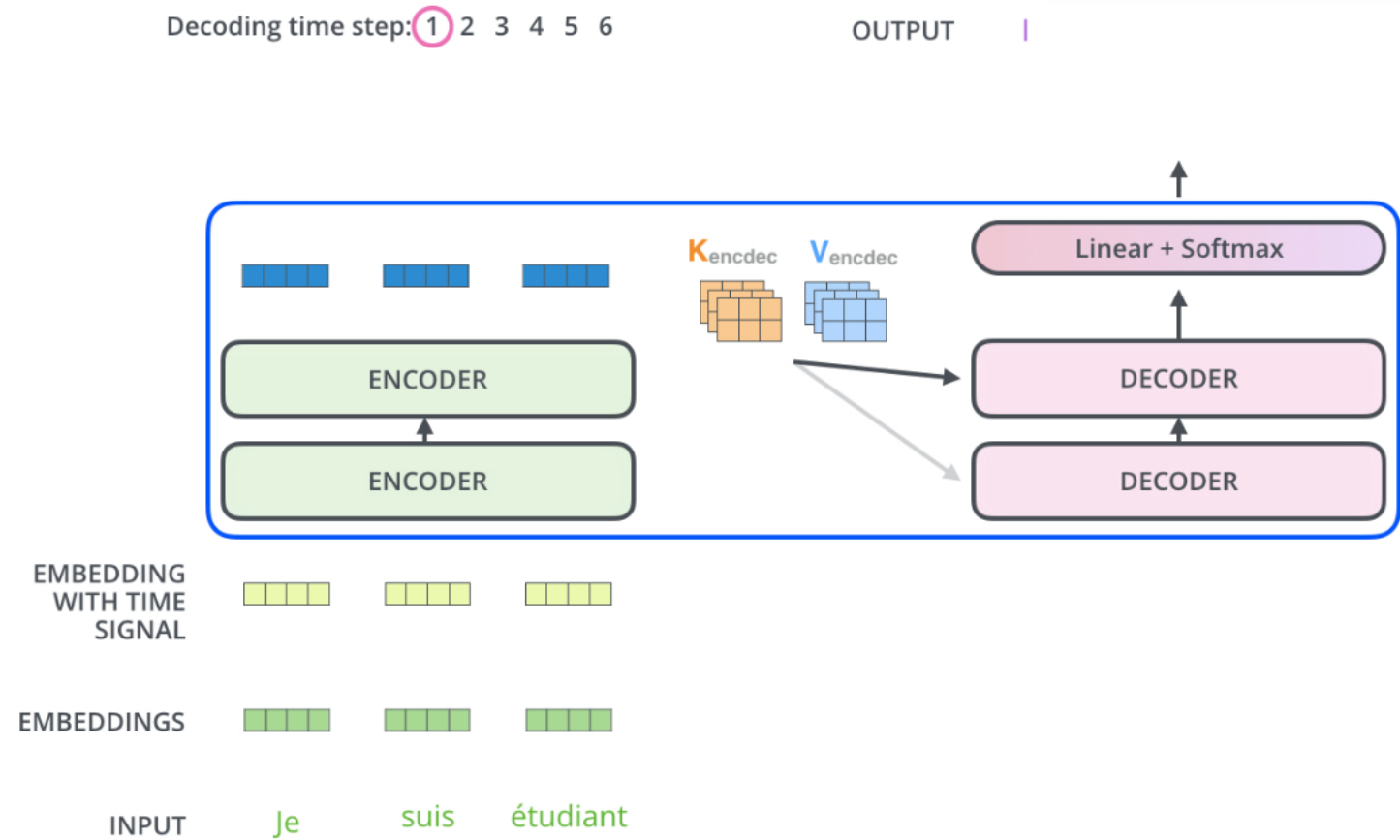
Se le pasa los **valores del último encoder** al decoder
→ que va a obtener la información relevante de esos valores usando también capas de atención y FNN

- El decoder recibe Q ("¿A quién debo prestar atención?) del decoder anterior
- V ("Información que apporto") y K ("¿Qué tan relevante soy?") vienen del encoder



Proceso de los decoders

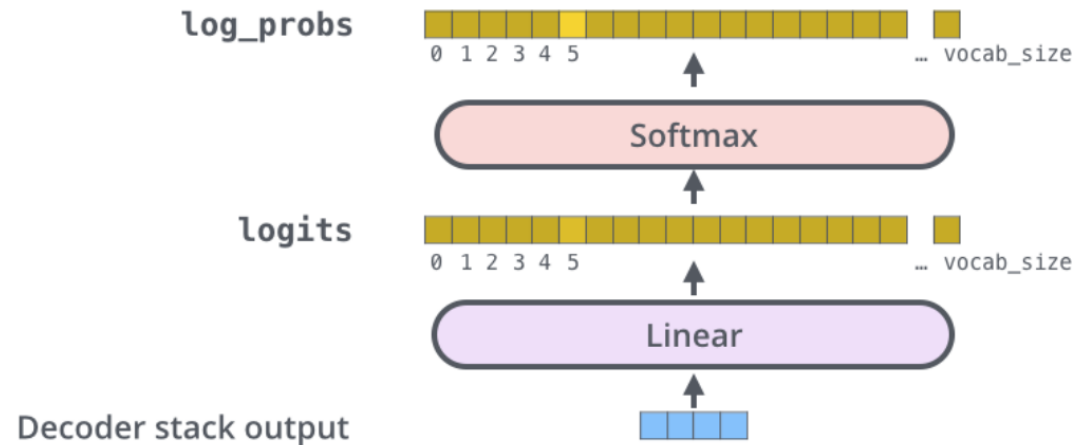
- La salida del encoder se **transforma en K y V** y se le pasa a los decoders



Última capa – Lineal + SoftMax

Which word in our vocabulary
is associated with this index?

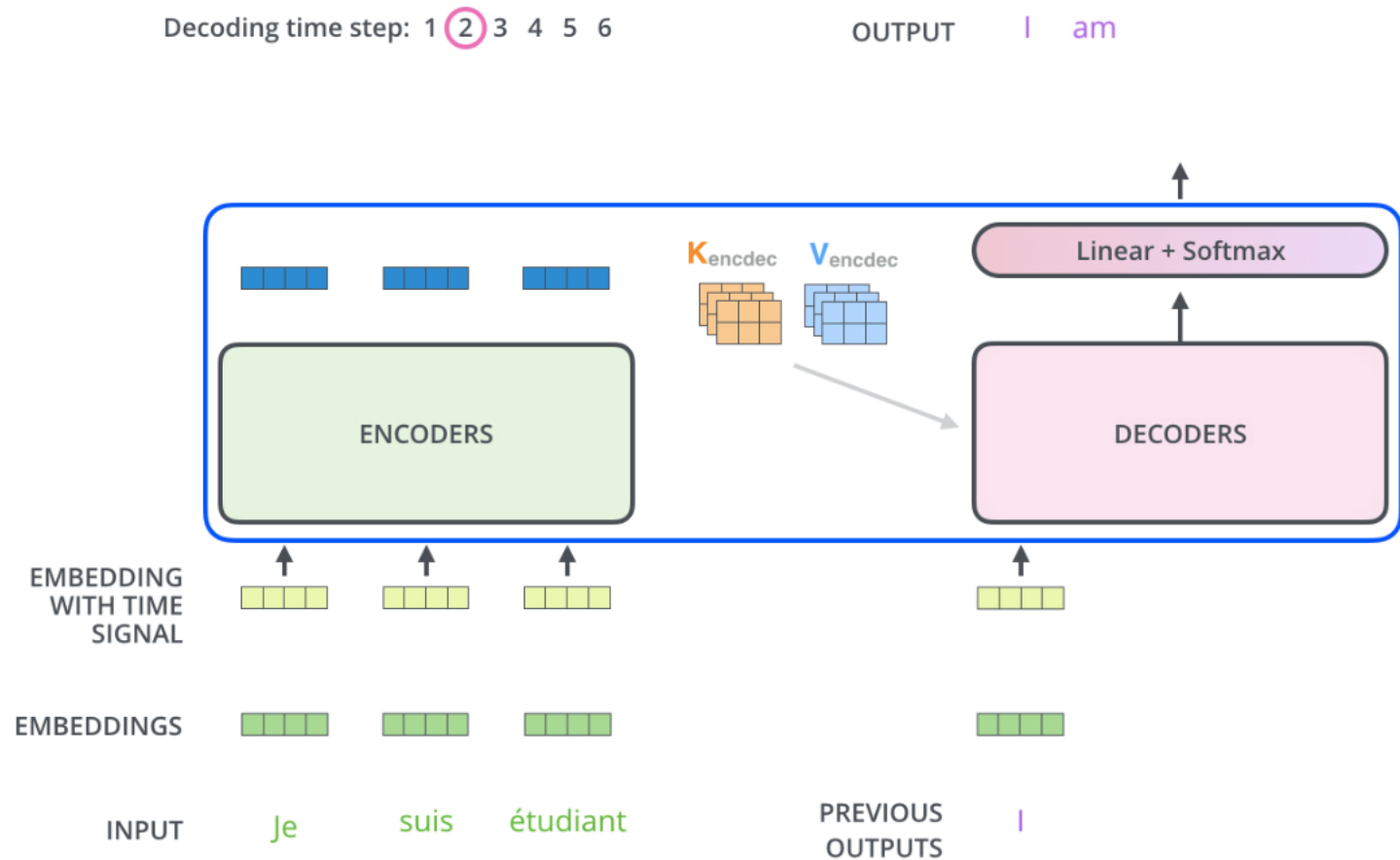
Get the index of the cell
with the highest value
(argmax)



Transforman el vector
obtenido en el decoder en
la palabra final

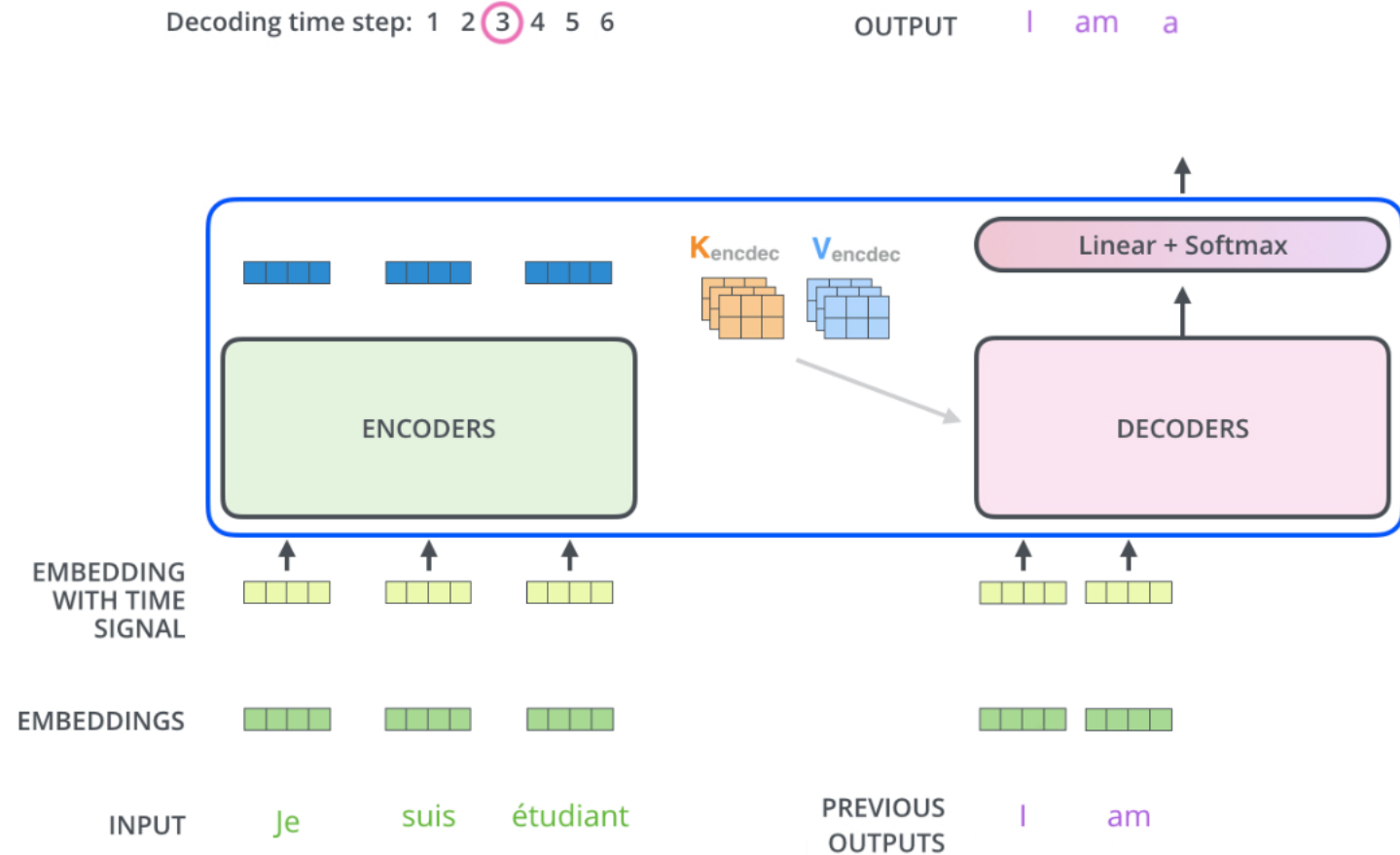
Proceso de los decoders

- Sigüientes iteraciones: se usan las salidas anteriores como Q
- K y V siguen obteniéndose del encoder



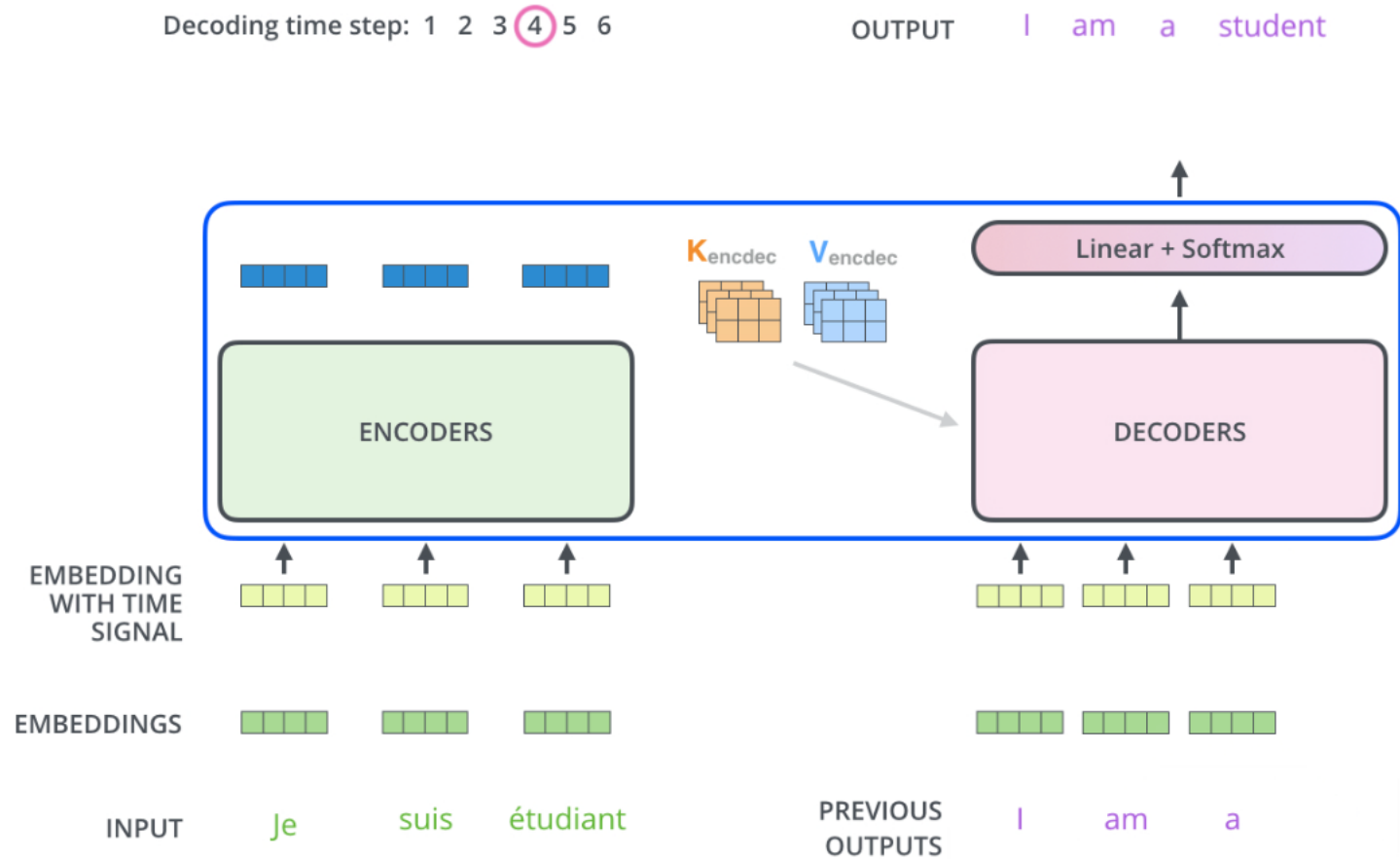
Proceso de los decoders

Siguientes iteraciones: se usan las salidas anteriores



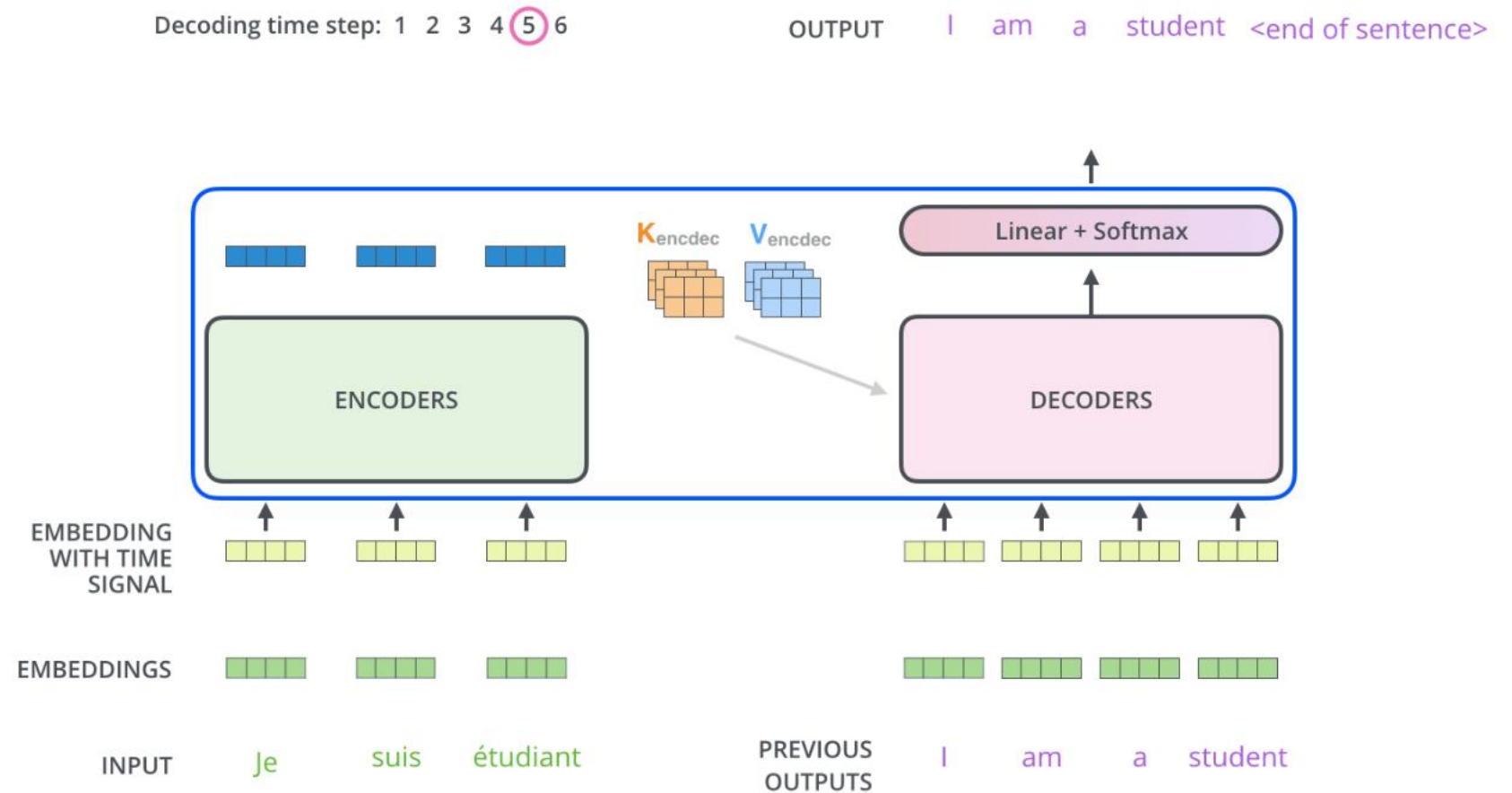
Proceso de los decoders

Siguientes iteraciones: se usan las salidas anteriores

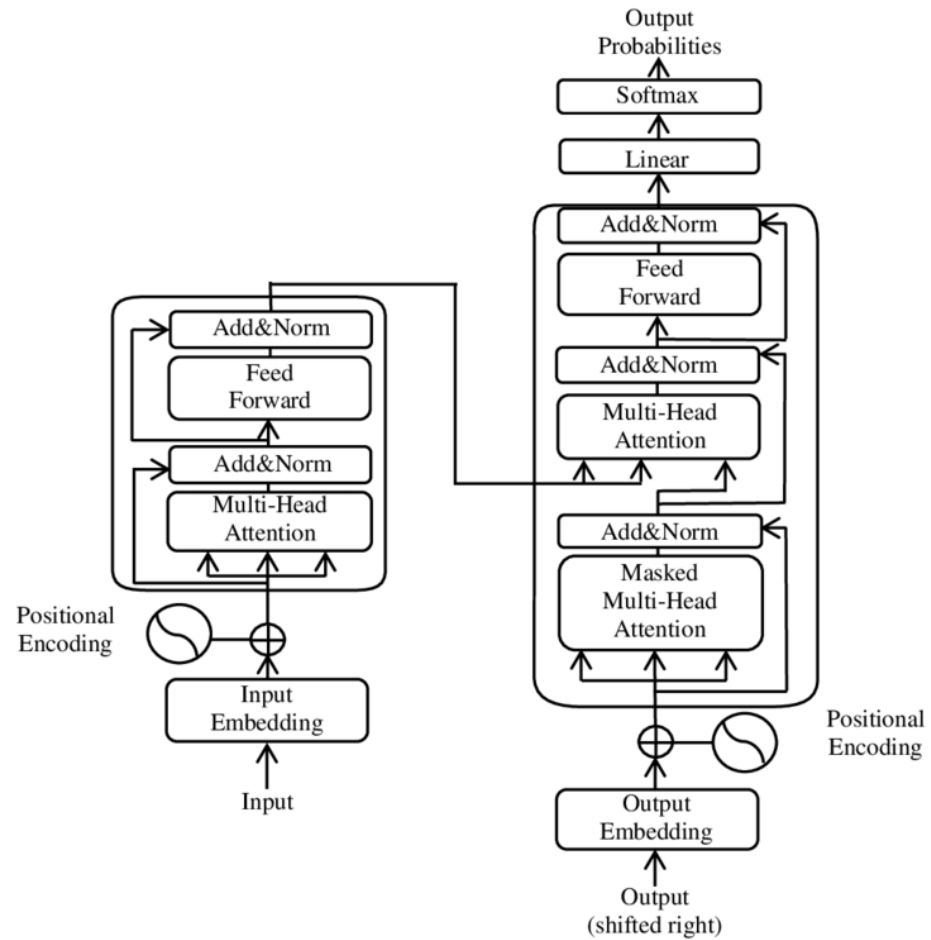


Proceso de los decoders

Siguientes iteraciones: se usan las salidas anteriores



Arquitectura Transformers



Modelos que usan transformers

Los más conocidos:

- **BERT (Bidirectional Encoder Representations from Transformers):** BERT es un modelo pre entrenado desarrollado por Google. Utiliza una arquitectura de codificador Transformer para capturar la **bidireccionalidad del contexto** en las representaciones de palabras
 - Semántica de textos
- **GPT (Generative Pre-trained Transformer):** GPT es una serie de modelos pre entrenados desarrollados por OpenAI que se utilizan para tareas de generación de texto. GPT utiliza una arquitectura de decodificador Transformer y se **entrena para predecir la siguiente palabra en una secuencia de texto dada una entrada previa**
 - Generación de textos

Large Language Models (LLMs)

LLMs

- LLMs o Large Language Model es un modelo de lenguaje grande que utiliza Deep Learning, normalmente Transformers
- Elementos básicos de una interacción con un LLM:
 - Prompts
 - Respuestas



Tipos de respuestas en LLMs

- **Fact-based Answers:** respuestas directas a preguntas (factual questions).
 - Ejemplo: ¿cual es la capital de Francia? → París
- **Decision making:** Ayuda a elegir entre opciones, analizando el contexto y los criterios que se le pide.
 - Ejemplo: decidir entre 2 estrategias comerciales.
- **Analysis:** descomponer información compleja en conocimiento.
 - Ejemplo: analizar tendencias de mercado basado en datos recientes.

Tipos de respuestas en LLMs

- **Recomendaciones:** ofrece sugerencias personalizadas basadas en preferencias.
 - Ejemplo: ¿Puedes recomendar un buen libro sobre IA? → selecciona una lista de libros
- **Resumen.** Condensa textos largos en resúmenes concisos.
 - Ejemplo: resumir un artículo largo teniendo en cuenta los puntos clave.
- **Traducción:** convierte un texto desde un idioma a otro.
 - Ejemplo: hacer información accesible a otros idiomas.

Ejemplos de tipos de respuestas

- *Ejemplo 1 - Prompt:* dime los puntos más destacables de la trayectoria profesional de Taylor Swift
- *Ejemplo 1 - Tipos de respuesta:*
 - Podría parecer solo fact-based
 - También tiene análisis: de toda la información de la que dispone sobre Taylor, va a analizarla y responder solo los puntos destacables
- *Ejemplo 2 - Prompt:* dame el número total de medallas de España en los juegos olímpicos modernos
- *Ejemplo 2 - Tipos de respuesta:*
 - Fact-based
 - Análisis: suma la información de la base de datos en oro, plata, bronce, y el total, etc.

Optimización de respuestas en LLMs

- Ajustando los parámetros del modelo LLM
- Importante: conocer el concepto de **token**

Token:

- Son las piezas más pequeñas de un texto procesado por el modelo.
- El token y sus tipos dependen de la estrategia de tokenización. Pueden ser palabras enteras, subpalabras o caracteres.

Ejemplo: tokens en chatgpt

- En este enlace: <https://platform.openai.com/tokenizer> Se pueden ver los token que usaría chatgpt en un texto dado.



The screenshot shows the OpenAI tokenizer interface. At the top, there are three tabs: "GPT-4o & GPT-4o mini" (selected), "GPT-3.5 & GPT-4", and "GPT-3 (Legacy)". Below the tabs is a text input area containing the text: "OpenAI's large language models process text using tokens, which are common sequences of characters found in a set of text. The models learn to understand the statistical relationships between these tokens, and excel at producing the next token in a sequence of tokens." Below the input area are two buttons: "Clear" and "Show example". Below the buttons is a table showing the tokenization results:

Tokens	Characters
49	268

Below the table is a visual representation of the text with colored blocks representing individual tokens. The text is: "OpenAI's large language models process text using tokens, which are common sequences of characters found in a set of text. The models learn to understand the statistical relationships between these tokens, and excel at producing the next token in a sequence of tokens."

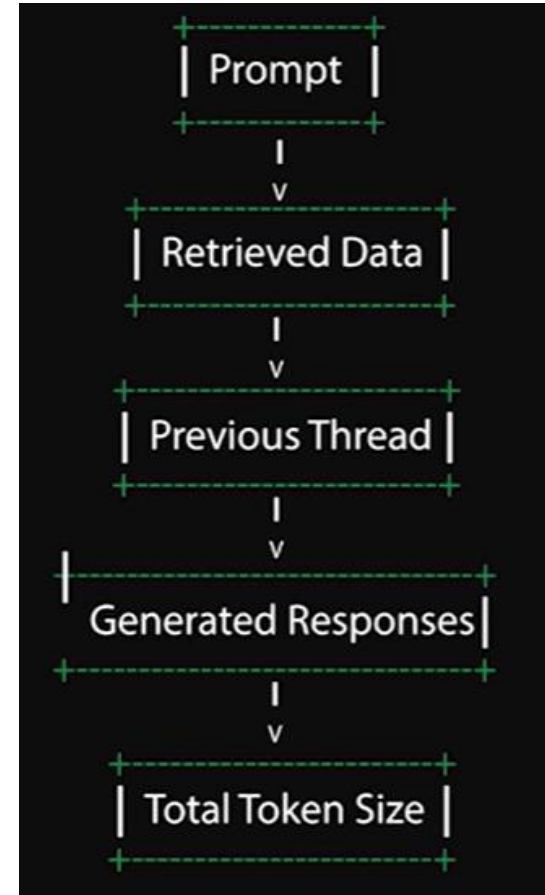
Como se ve, casi siempre son palabras. aunque a veces no tiene por qué, por ejemplo, aquí hay comas, o 's.

Comprender cómo funcionan los tokens es fundamental para optimizar el rendimiento del modelo y garantizar que las respuestas sean coherentes y relevantes

Tamaño del token

Tamaño total del token =

- tamaño del prompt +
- el tamaño de los datos obtenidos de ese prompt +
- el tamaño del hilo previo (preguntas previas que le hemos hecho) +
- las respuestas generadas a esas preguntas



Context window

- Context window o ventana de contexto: La cantidad máxima de texto, medida en tokens, que el LLM puede considerar al mismo tiempo al generar una respuesta
 - Es como una memoria a corto plazo
- El tamaño de la ventana de contexto es limitante para obtener las mejores respuestas.
- La ventana de contexto no incluye el total de tokens que tenemos, sino que solo abarcará una parte.
- El tamaño de la ventana dependerá del modelo.
 - Cuanto más grande y costoso sea el modelo, más grande será la ventana también.

Fine-tuning en LLMs

- Encontrar los mejores valores de los parámetros del LLM para obtener mejores resultados

Parámetros a configurar:

- **La ventana de contexto.** Una ventana más grande mejora la consistencia en conversaciones largas.
- **Max Output token limit:** el máximo número de token que el modelo puede generar.
 - Para obtener respuestas concisas y manejables

Fine-tuning en LLMs

Parámetros a configurar:

- **Temperatura.** Controla la aleatoriedad del output del modelo.
 - Cuanto más bajos sean los valores de temperatura, más deterministas y precisas son las respuestas.
 - Cuanto más altos, más aleatoriedad, lo que puede ser útil para obtener respuestas creativas y diversas.
- **Stop sequences.** Le dice al modelo cuando parar de generar el output usando un conjunto de secuencias, por lo que controla la longitud y el final de la respuesta generada.
 - Cuando encuentra una de las secuencias, para de generar la respuesta

Fine-tuning en LLMs

- **Top P (Nucleus Sampling).** El LLM considera solo los tokens con una probabilidad acumulada por encima de un threshold p .
 - Ayuda a generar respuestas más acordes al contexto enfocándose en el subconjunto de los token con más importancia y significado para obtener respuestas coherentes con respecto a las palabras anteriores.
- **Frequency penalty.** Reduce la probabilidad de generar tokens usados frecuentemente en el texto.
 - Evita la repetición y mantiene la diversidad del vocabulario.

Fine-tuning en LLMs

- **Presence penalty.** Reduce la probabilidad de repetir tokens ya generados previamente.
 - Asegura que el texto generado es variado y nuevo.
 - La diferencia con la anterior, es que esta se refiere a los tokens ya generados en la respuesta (ese hilo). La de frequency es en general texto frecuente.

Modelos en OpenAI y Gemini

- OpenAI: <https://platform.openai.com/docs/models>
- Gemini: <https://ai.google.dev/gemini-api/docs/models?hl=es-419>

Ingeniería de Prompts + In-Context Learning (ICL)

Ingeniería de Prompts

- **Ingeniería de prompts:**
 - Metodología para diseñar el texto de entrada (prompt) a dar a un LLM para optimizar la respuesta del modelo
 - Conocimiento/habilidad humana
- **In-context learning:** capacidad de la máquina de aprender a partir del prompt
 - Cuánto mejor sea la ingeniería de prompts → mejor in-context learning

Ingeniería de Prompts

- Los prompts deberían tener 4 componentes
- Debemos tenerlo en cuenta para construir el prompt

Componentes:

- **Instrucción:** configura el rol que tiene que tomar el modelo que responde
- **Contexto:** incluye información necesaria para entender la situación en la que se necesita la solución
- **Pregunta:** la parte más importante del prompt → lo que necesitamos que el modelo responda
- **Formato de output:** el formato y los detalles específicos de lo que se necesita en la respuesta

Ejemplo de prompt

Situación: una empresa se va a mover a una oficina, y los empleados tienen que hacer un montón de papeleo y llevar a cabo muchos trámites, además de la mudanza física. Necesitamos definir el protocolo de actuación.

- **Instrucción:** Eres el responsable de políticas de recursos humanos
- **Contexto:** La oficina se va a mover a una nueva localización, y los empleados necesitan ayuda con el proceso de mudanza y traslado
- **Pregunta:** ¿Cuál es la política de traslado a llevar a cabo paso por paso?
- **Formato de output:** incluye los pasos, los enlaces a los formularios, los contactos de las personas y los deadlines para cada uno de los pasos

Estrategias de prompting

- Métodos para crear prompts
- Objetivo: reducir alucinaciones

Los más típicos (pero hay más):

- Zero-Shot
- Few-shot
- Chain of Thoughts

Few-shot + Chain-of-thoughts → muy potente

Zero-shot prompting

- Solo se hace la pregunta (incluyendo, rol, contexto y output)
- Peor método: no reduce alucinaciones
- Ejemplo - *Prompt*:
 - Eres un analista financiero experto.
 - Analiza el siguiente caso y determina si la empresa debería invertir en el proyecto.
 - Considera riesgos, retorno esperado y factores estratégicos.
 - Da una recomendación final justificada en menos de 150 palabras.

Caso:

Una empresa tecnológica puede invertir 2 millones de euros en desarrollar un nuevo producto de IA.

Existe un 60% de probabilidad de generar 5 millones en ingresos en 3 años,
un 30% de probabilidad de generar solo 1 millón,
y un 10% de probabilidad de fracaso total.

El mercado está creciendo rápidamente, pero hay competidores fuertes.

Few-shot prompting

- En el prompt se incluyen ejemplos
- Ejemplo - *Prompt*:

Eres un analista financiero experto. Analiza cada caso y da una recomendación final.

Ejemplo 1:

Caso:

Una empresa puede invertir 100.000€ con un 90% de probabilidad de ganar 150.000€ y un 10% de perderlo todo.

Recomendación: Invertir.

Justificación: El valor esperado es positivo y el riesgo es bajo.

Ejemplo 2:

Caso:

Una empresa puede invertir 500.000€ con un 20% de probabilidad de ganar 2 millones y un 80% de perderlo todo.

Recomendación: No invertir.

Justificación: El riesgo es demasiado alto para la probabilidad de éxito.

Few-shot prompting

- En el prompt se incluyen ejemplos
- Ejemplo – *Prompt (continuación)*:

Ahora analiza:

Caso:

Una empresa tecnológica puede invertir 2 millones de euros en desarrollar un nuevo producto de IA.

Existe un 60% de probabilidad de generar 5 millones en ingresos en 3 años, un 30% de probabilidad de generar solo 1 millón, y un 10% de probabilidad de fracaso total. El mercado está creciendo rápidamente, pero hay competidores fuertes.

Chain-of-Thought

- Se pide el razonamiento paso a paso
- Ejemplo – *Prompt*:

Eres un analista financiero experto. Analiza el siguiente caso paso a paso antes de dar una recomendación final.

Incluye:

1. Cálculo del valor esperado
2. Evaluación del riesgo
3. Factores estratégicos
4. Conclusión final

Caso:

Una empresa tecnológica puede invertir 2 millones de euros en desarrollar un nuevo producto de IA. Existe un 60% de probabilidad de generar 5 millones en ingresos en 3 años, un 30% de probabilidad de generar solo 1 millón, y un 10% de probabilidad de fracaso total.

El mercado está creciendo rápidamente, pero hay competidores fuertes.

Chain-of-Thought con demostración

- Se le da un ejemplo de razonamiento paso a paso
- Ejemplo – *Prompt*:

Caso:

Una empresa puede invertir 100.000€ con un 50% de probabilidad de ganar 200.000€ y un 50% de perderlo todo.

Razonamiento:

1. Si gana, el beneficio neto es 100.000€ (200.000 - 100.000).
2. Si pierde, el resultado neto es -100.000€.
3. Valor esperado = $(0,5 \times 100.000) + (0,5 \times -100.000) = 0€$.
4. El valor esperado es neutro, pero existe riesgo significativo sin ventaja clara.

Respuesta final:

No invertir.

Chain-of-Thought con demostración

- Se le da un ejemplo de razonamiento paso a paso
- Ejemplo – *Prompt (continuación)*:

Ahora analiza el siguiente caso:

Caso:

Una empresa tecnológica puede invertir 2 millones de euros en desarrollar un nuevo producto de IA.

Existe un 60% de probabilidad de generar 5 millones en ingresos en 3 años, un 30% de probabilidad de generar solo 1 millón, y un 10% de probabilidad de fracaso total.

El mercado está creciendo rápidamente, pero hay competidores fuertes.

Retrieval Augmented Generation (RAG)

Retrieval Augmented Generation (RAG)

Es una técnica de IA que combina:

- **Information retrieval:** el sistema busca información relevante en fuentes de conocimiento externas:
 - bases de conocimiento, bases de datos, documentación...
- **Generación de texto:** un modelo de lenguaje usa la información recuperada para crear una respuesta
 - En la actualidad, se hace con LLMs

Las respuestas no se basan únicamente en datos estáticos preentrenados → incluyen información actual y específica → mejora la precisión y la utilidad

Retrieval Augmented Generation (RAG)

Proceso paso a paso:

1. Inicio
2. Recibir query o prompt
3. Recuperar información relevante de la base de datos / base de conocimiento
4. Acceder a información actualizada y específica
5. Usar la información recuperada como contexto para añadir al prompt
6. Generar respuesta utilizando un LLM
7. Proporcionar respuesta final
8. Fin

Retrieval Augmented Generation (RAG)

Ventajas:

- Reduce alucinaciones
- Genera respuestas más exactas
- Respuestas más actualizadas
- Se puede hacer uso de datos confidenciales o privados
- Es un planteamiento más transparente

Retrieval Augmented Generation (RAG)

Las bases de conocimiento pueden ser métodos o técnicas de IA.
Ejemplo: grafos de conocimiento o CBR

Framework muy usado para implementar RAGs: LangChain
(<https://github.com/langchain-ai/langchain>)

- APIs generales de LLMs → solo permiten consultas haciendo prompts
- LangChain permite hacer encadenamiento de operaciones e integración con fuentes de datos externas → implementación de RAGs